



Software Engineering

Unit 4: Lecture 1

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof. Rajesh Banginwar
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

SW Quality and Testing: Metrics and Measurement

Course Timeline

WEEK	UNIT 1				UNIT 2		UNIT 3			UNIT 4			
	1	2	3	4	5	6	7	8	9	10	11	12	13
TOPICS	SDLC	REQUIREMENTS		Proj Management	ARCHITECTURE & DESIGN		SE & AI	Implementation, SCM, CI/ CD, Testing		Security/Privacy, OSS, etc			Project Demo & presentation
MAJOR PROJ MILESTONES	Team formation	Proj SRS		Complete Jira Backlog Project Test Plan	S/W Arch doc, S/W Des Doc		Sprint 1 (Implementation +Test Case Dev)		Sprint 2 (Implementation +Test Case Dev)		All test reports done		All deliverables + Demo
LABS			LAB 1, 2		LAB 3		LAB 4		LAB 5		LAB 6	LAB 7	



Today's Learning Goals

- Explain the importance of measurement and metrics in Software Engineering
- Provide examples of metrics for software qualities and process
- Apply goal-based frameworks for decision making using metrics
- Identify the limitations and dangers of decisions and incentives based on measurements

Measurement in everyday life

- Economics
 - price, inflation rate, stock price, volume
- Medicine
 - heart rate, blood pressure, body temperature, ECG
- Engineering
 - Force, torque, heat transfer coefficient, thermal efficiency
- Natural sciences
 - AQI, carbon footprint, Soil pH

NS New Scientist

Ants use pedometers to find home

An experiment that involves attaching stilts to ants' legs reveals that the insects somehow keep a record of how many steps they take.

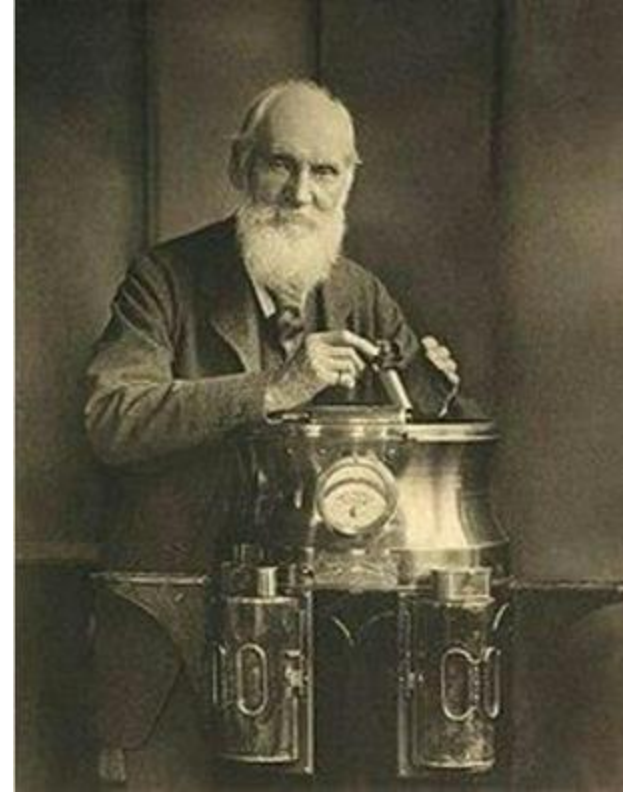
Jun 29, 2006



*“To measure is to know;
if you can not measure it,
you can not improve it”*

~William Thomson, Lord Kelvin

$$K = \left(\frac{5}{9} (F - 32) \right) + 273.15$$



Software Development...before Software Engineering



Software Engineering: Principles, practices (technical and non-technical) for confidently building high-quality software.

What does this mean?
How do we know?
Measurement and
metrics are **key**
concerns.

What is Measurement?

- Measurement is the empirical, objective assignment of numbers, according to a rule derived from a model or theory, to attributes of objects or events with the intent of describing them. – Craner, Bond, “Software Engineering Metrics: What Do They Measure and How Do We Know?”
- A quantitatively expressed reduction of uncertainty based on one or more observations. – Hubbard, “How to Measure Anything ...”

Software Quality Metrics

IEEE 1061 definition: “A software quality metric is a function whose inputs are software data and whose output is a single numerical value that can be interpreted as the degree to which the software possesses a given attribute that affects its quality.”

Entity

Object or
Process

Attribute

Quality of
Interest

Measurement

Method to obtain a
number or a symbol

What entities to we care about? (examples)

- Software product
- Modules
- Software development process
- People

What software qualities do we care about? (examples)

- Functionality (e.g., data integrity)
- Installability
- Availability
- Consistency
- Portability
- Scalability
- Security
- Extensibility
- Bugginess
- Regulatory compliance
- Documentation
- Performance

What process qualities do we care about? (examples)

- Development efficiency
- Meeting efficiency
- Conformance to processes
- Reliability of predictions
- Fairness in decision making
- Regulatory compliance
- On-time release

What people qualities do we care about? (examples)

➤ **Developers**

- Maintainability
- Performance
- Employee satisfaction and well-being
- Communication and collaboration
- Efficiency and flow
- Satisfaction with engineering system
- Regulatory compliance

➤ **Customers**

- Satisfaction
- Ease of use
- Feature usage
- Regulatory compliance

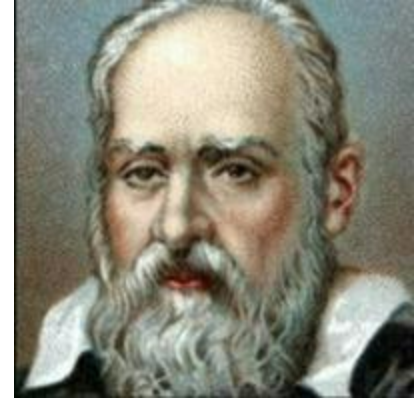
Non-trivial Qualities

- Software
 - Code elegance
 - Code maintainability
- Process
 - Fairness in decision making
- Team
 - Team collaboration
 - Creativity



*"Measure what is measurable, and
make measurable what is not so."*

~Galileo Galilei



Everything is measurable

- If X is something we care about, then X, by definition, must be detectable.
- How could we care about things like “quality,” “risk,” “security,” or “public image” if these things were totally undetectable, directly or indirectly?
- If we have reason to care about some unknown quantity, it is because we think it corresponds to desirable or undesirable results in some way.
- If X is detectable, then it must be detectable in some amount.
- If you can observe a thing at all, you can observe more of it or less of it
- If we can observe it in some amount, then it must be measurable.

Example: Code Complexity

Lines of Code

- Easy to measure

```
> wc -l file1  
file2....
```

LOC	projects
450	Expression Evaluator
2,000	Sudoku
100,000	Apache Maven
500,000	Git
3,000,000	MySQL
15,000,000	gcc
50,000,000	Windows 10
2,000,000,000	Google (MonoRepo)

Normalizing Lines of Code

- Ignore comments and empty lines
- Ignore lines < 2 characters
- Pretty print source code first
- Count statements (logical lines of code)

```
for (i = 0; i < 100; i += 1) printf("hello"); /* How many lines of code is this? */
```

```
/* How many lines of code is this? */
```

```
for (  
    i = 0;  
    i < 100;  
    i += 1  
) {  
    printf("hello");  
}
```

Normalizing By Language

Language	Statement factor (productivity)	Line factor
C	1	1
C++	2.5	1
Fortran	2	0.8
Java	2.5	1.5
Perl	6	6
Smalltalk	6	6.25
Python	6	6.5

Halstead Volume

- Introduced by Maurice Howard Halstead in 1977
- Halstead Volume =
 number of operators/operands *
 $\log_2$ (number of distinct operators/operands)
- Approximates size of elements and vocabulary

Cyclomatic Complexity

- Proposed by McCabe 1976
 - Based on control flow graph, measures linearly Independent paths through a program
 - $\sim$ = number of decisions
 - Number of test cases needed to achieve branch coverage
- $M = \text{edges of CFG} - \text{nodes of CFG} + 2 * \text{connected components}$

```
if (c1) {  
    f1();  
} else {  
    f2();  
}  
if (c2) {  
    f3();  
} else {  
    f4();  
}
```

“For each module, either limit cyclomatic complexity to [X] or provide a written explanation of why the limit was exceeded.”
– NIST Structured Testing methodology

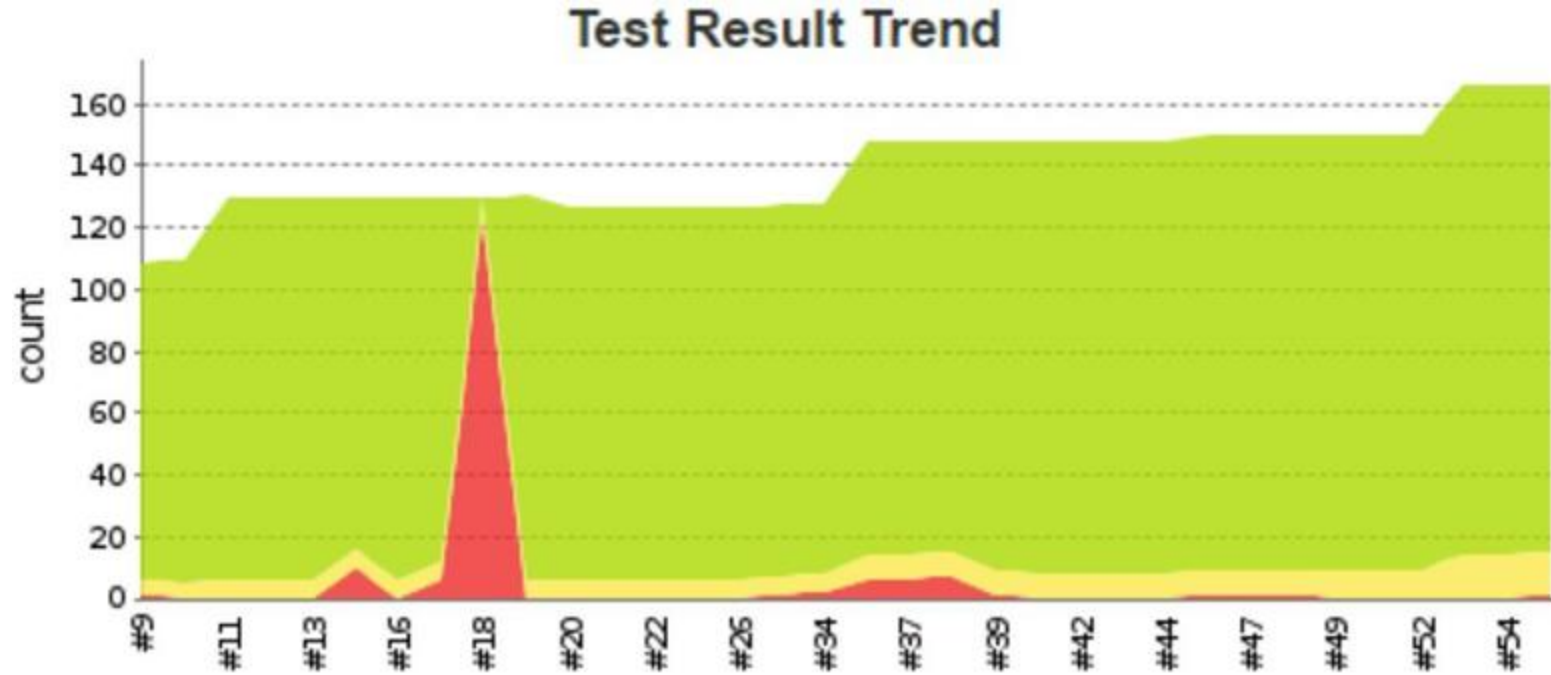
Object-Oriented Metrics

- Number of Methods per Class
- Depth of Inheritance Tree
- Number of Child Classes
- Coupling between Object Classes
- Calls to Methods in Unrelated Classes

Measurement for Decision Making

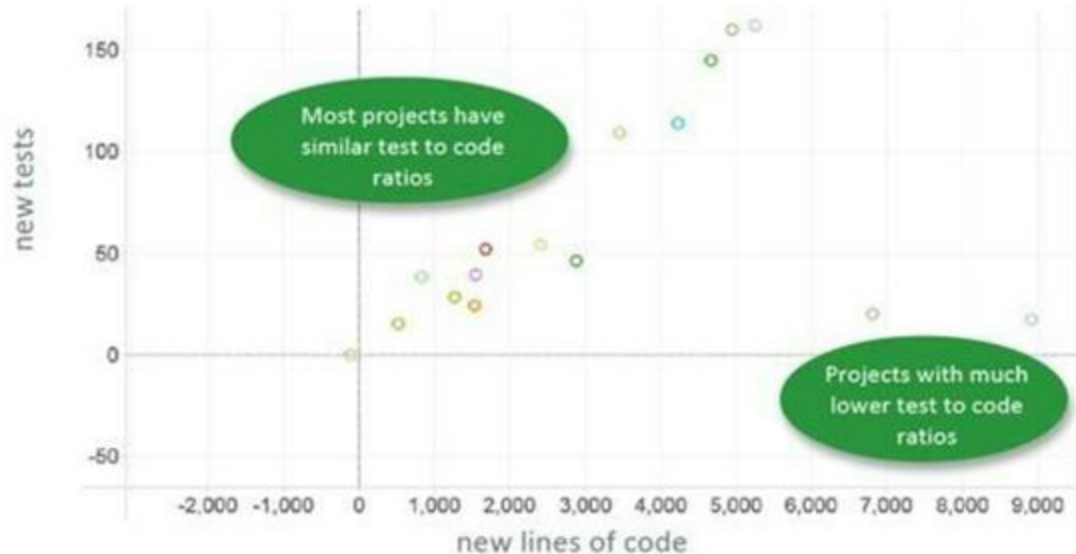
- Fund project?
- More testing?
- Fast enough? Secure enough?
- Code quality sufficient?
- Which feature to focus on?
- Developer bonus?
- Time and cost estimation? Predictions reliable?

Trend analyses



Benchmarking against standards

- Monitor many projects or many modules, get typical value for metrics
- Report deviations



Test Measurements

Software Test Related Measurements and Metrics

Software Test Metric: Quantitative Measure of testing process indicating progress, quality, productivity and degree to which system possesses a given attribute

Goal

- Improve efficiency and effectiveness of software testing process
- Make better decisions for future testing process using reliable data

Measurements: critical in performing SQA and testing

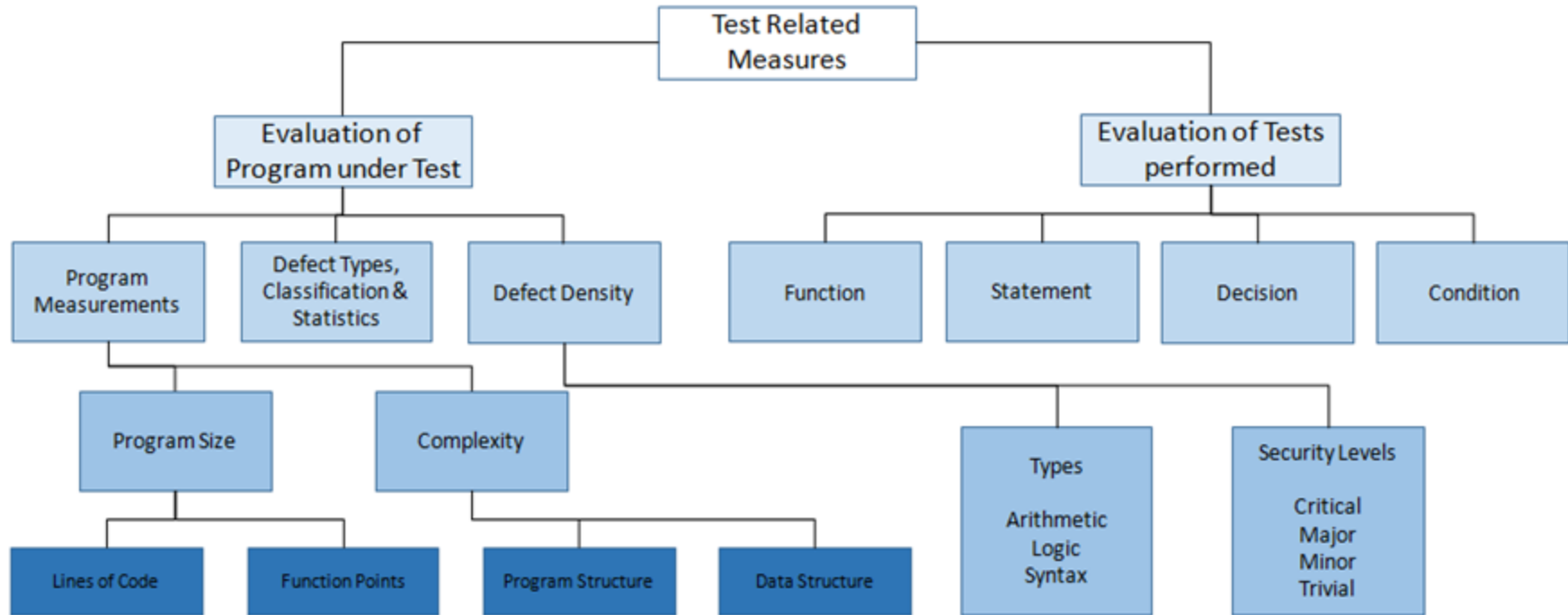
Support increasing effectiveness of testing process

- when used as part of project planning and monitoring
- evaluate the quality of product under test
- helps in defect analysis

Measures can

- represent the quality
- represent test coverage

Software Test Measurements



Function Points

- Function points measure the size of an application system based on the functional view of the system. The size is determined by counting the number of inputs, outputs, queries, internal files and external files in the system and adjusting that total for the functional complexity of the system.
- Function point analysis, originally developed at IBM, has as an advantage its focus on measuring software produced in terms of functionality delivered to the end user, rather than in terms of development deliverables, which have no direct bearing on the end user.

Function Points

FP Attribute Types or Information Domain Characteristics.

Measurements Parameters	Examples
1.Number of External Inputs(EI)	Input screen and tables
2. Number of External Output (EO)	Output screens and reports
3. Number of external inquiries (EQ)	Prompts and interrupts.
4. Number of internal files (ILF)	Databases and directories
5. Number of external interfaces (EIF)	Shared databases and shared routines.

Function Points

Weights of 5-FP Attributes

Measurement Parameter	Low	Average	High
1. Number of external inputs (EI)	7	10	15
2. Number of external outputs (EO)	5	7	10
3. Number of external inquiries (EQ)	3	4	6
4. Number of internal files (ILF)	4	5	7
5. Number of external interfaces (EIF)	3	4	6

Software Test Related Measures – Product Related

Metrics	Description
SLOC	Size in Lines of Code
Fault Density	Ratio of number of faults found to size of the programs
MTBF	Mean Time Between Failure is based on statistical analysis to indicate probability of failure
Failure Rate	Inverse of MTBF
Defect Distribution	% of defects attributed to a specified phase in SDLC
Defect Density of Modules	Ratio of number of faults found in module to total faults found in product
Defect Leakage	Test Efficiency = $\left[\frac{\text{Total number of defects found in UAT}}{\text{Total number of defects found before UAT}} \right] \times 100$

- Measurements and Metrics
- How to use measurements and metrics?
- Case study: Autonomous Vehicle Software
- Risks and challenges
- Metrics and incentives

AV Software is ...



By what metrics can we judge AV software(e.g., safety)??

(1) Code coverage

- Amount of code executed during testing.
- Statement coverage, line coverage, branch coverage, etc.

E.g. : 75% branch coverage i.e 3/4 if-else outcomes have been executed

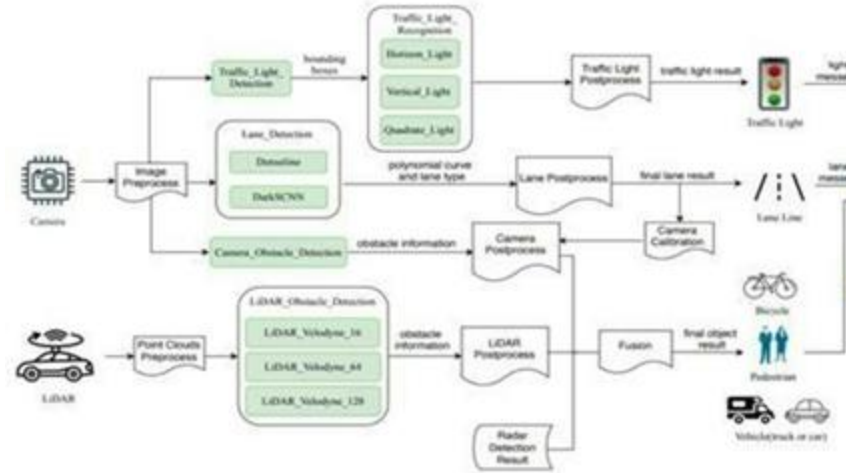
```

1698 : const TrajectoryPoint& StGraphData::init_point() const { return init_point_; }
2264 : const SpeedLimit& StGraphData::speed_limit() const { return speed_limit_; }
212736 : double StGraphData::cruise_speed() const {
212736 :     return cruise_speed_ > 0.0 ? cruise_speed_ : FLAG_default_cruise_speed;
}
1698 : double StGraphData::path_length() const { return path_data_length_; }
1698 : double StGraphData::total_time_by_conf() const { return total_time_by_conf_; }
1698 : planning_internal::STGraphDebug* StGraphData::mutable_st_graph_debug() {
1698 :     return st_graph_debug_;
}
546 : bool StGraphData::setSTDriveableBoundary(
    const std::vector<std::tuple<double, double, double>>& a_boundary,
    const std::vector<std::tuple<double, double, double>>& v_obs_info) {
[*] 546 :     if (a_boundary.size() != v_obs_info.size()) {
        return false;
}
40752 : for (size_t i = 0; i < a_boundary.size(); ++i) {
40752 :     auto st_bound_instance = st_driveable_boundary_.add_st_boundary();
40752 :     st_bound_instance->set_t(std::get<0>(a_boundary[i]));
40752 :     st_bound_instance->set_x_lower(std::get<1>(a_boundary[i]));
40752 :     st_bound_instance->set_x_upper(std::get<2>(a_boundary[i]));
40752 :     if (std::get<1>(v_obs_info[i]) > -kObsSpeedIgnoreThreshold) {
40752 :         st_bound_instance->set_v_obs_lower(std::get<1>(v_obs_info[i]));
}
40752 :     if (std::get<2>(v_obs_info[i]) < kObsSpeedIgnoreThreshold) {
40752 :         st_bound_instance->set_v_obs_upper(std::get<2>(v_obs_info[i]));
}
}
}

```

(2) Model Accuracy

- Train machine-learning models on labelled data (sensor data + ground truth).
- Compute accuracy on a separate labelled test set.
- E.g., 90% accuracy implies that object recognition is right for 90% of the test inputs.



(3) Failure Rate

- Frequency of crashes /fatalities
- Per 1,000 rides, per million miles, per month (in the news)

TRANSP / WAYMO / TECH

Waymo's driverless cars were involved in two crashes and 18 'minor contact events' over 1 million miles



Image: Allison J. Schwartz / Los Angeles Times

/ The Alphabet-owned company pulls back the curtain on more stats from its public road testing. Of the 20 incidents, only two met the federal government's reporting criteria, and no one was injured.

By Andrew J. Hawkins, transportation editor with 10+ years of experience who covers EVs, public transportation, and aviation. His work has appeared in The New York Times, Reuters, and CNN.

'Complete meltdown': Driverless cars in San Francisco stall causing a traffic jam

By Jessica Stenning, 1000 Business, September 9-10 PM EDT Mon August 14, 2017

f t i y

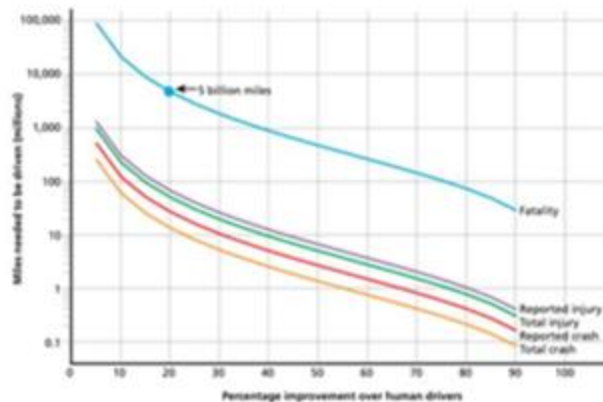


A Cruise autonomous car in San Francisco

(4) Mileage

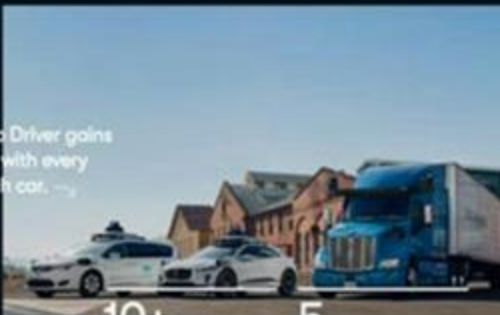


Figure 3. Miles Needed to Demonstrate with 95% Confidence that the Autonomous Vehicle Failure Rate Is Lower than the Human Driver Failure Rate



Building the World's Most Experienced Driver™

The Waymo Driver gains experience with every mile, in each car.



10+ More than a Decade of Autonomous Driving in More than 10 States	5 Generations of Autonomously Driven Vehicles
15+ Billion Autonomously Driven Miles in Simulation	20+ Million Real-World Miles on Public Roads

Outline

- Measurements and Metrics
- How to use measurements and metrics?
- Case study: Autonomous Vehicle Software
- Risks and challenges
- Metrics and incentives

The streetlight effect

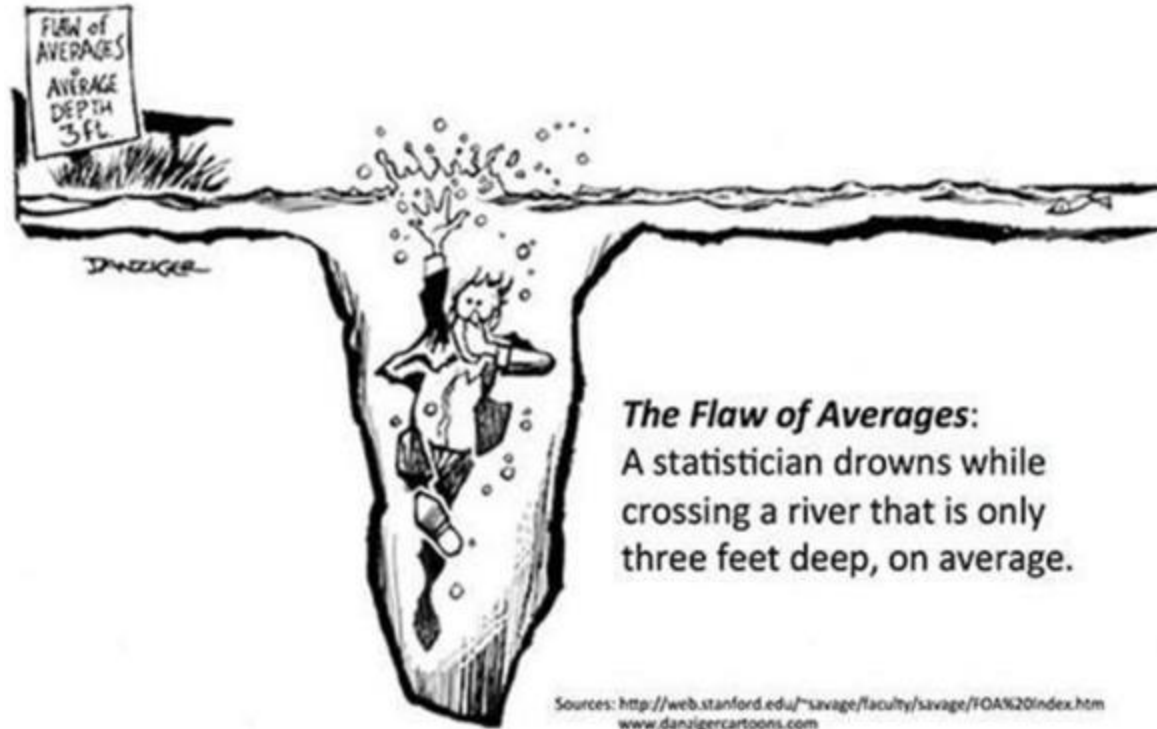


The streetlight effect

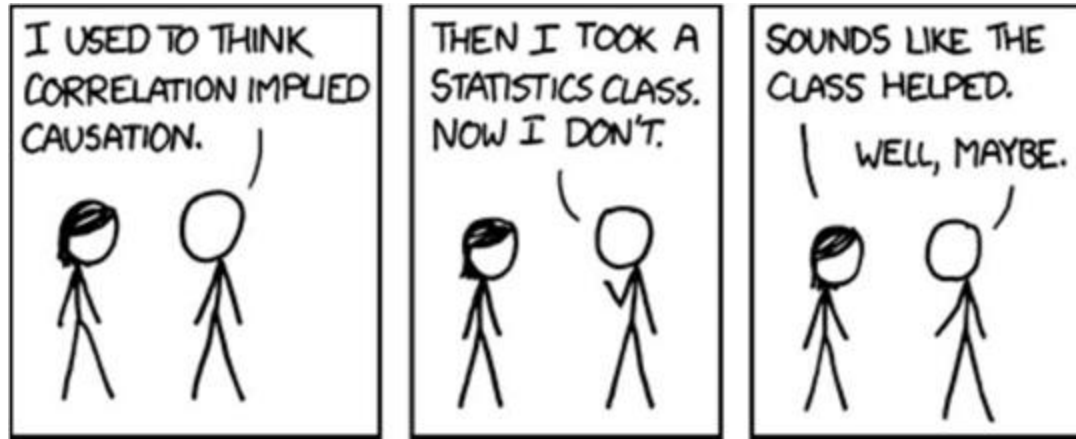


- A known observational bias.
- People tend to look for something only where it's easiest to do so.
- If you drop your keys at night, you'll tend to look for it under streetlights.

Bad statistics: What could possibly go wrong?

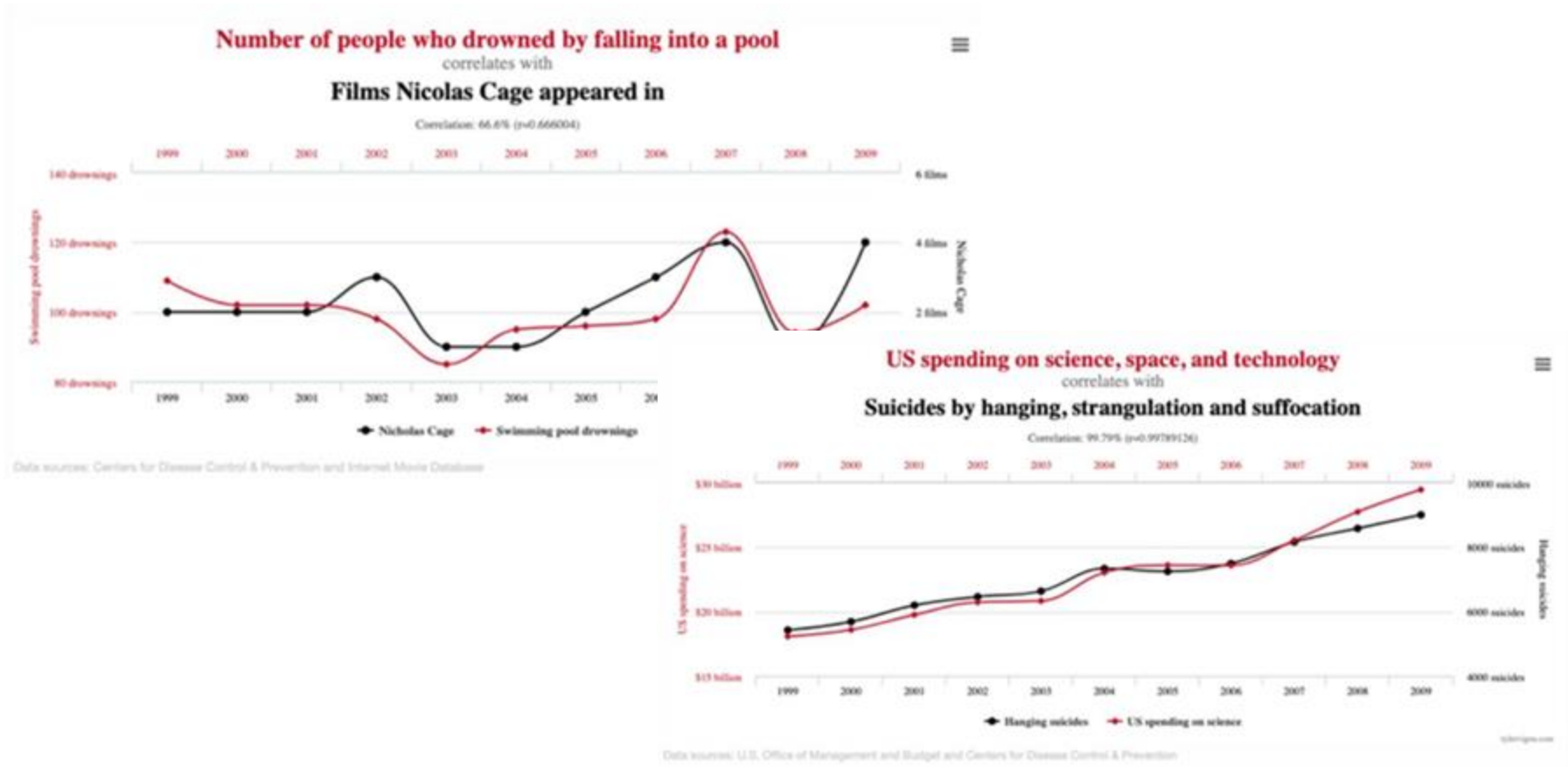


Making inferences

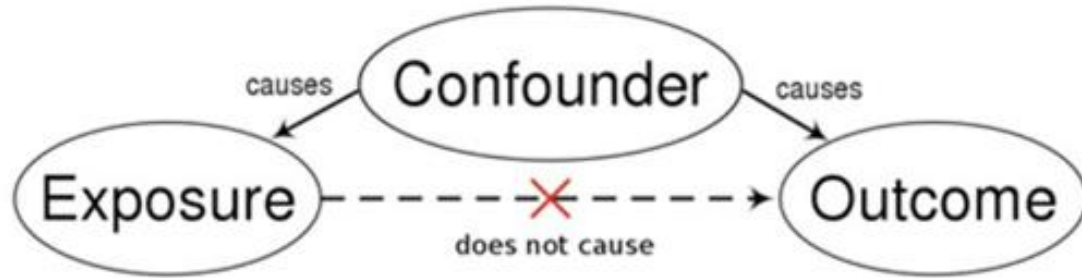
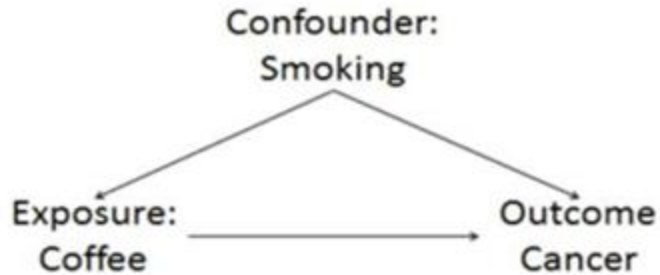


- Provide a theory (from domain knowledge, independent of data)
- Show correlation
- Demonstrate ability to predict new cases (replicate/validate)

Spurious Correlations



Confounding variables



- If you look only at the coffee consumption → cancer relationship, you can get very misleading results
- Smoking is a confounder

Confounding Variables

RESEARCH-ARTICLE

Coverage is not strongly correlated with test suite effectiveness

Authors:  [Laura Inozemtseva](#),  [Reid Holmes](#) [Authors Info & Affiliations](#)

ICSE 2014: Proceedings of the 36th International Conference on Software Engineering • May 2014 • Pages 435–445 • <https://doi.org/10.1145/2568225.2568271>

“We found that there is a low to moderate correlation between coverage and effectiveness when the number of test cases in the suite is controlled for.”

Most studies did not account for the confounding influence of test suite size.

Measurement Reliability

- Extent to which a measurement yields similar results when applied multiple times

- Goal is to reduce uncertainty, increase consistency
Example: Performance ,Time, memory usage, Cache misses, I/O operations, instruction execution count, etc.

- Law of large numbers
 - Taking multiple measurements to reduce error
 - Trade-off with cost

- Measurements and Metrics
- How to use measurements and metrics?
- Case study: Autonomous Vehicle Software
- Risks and challenges
- Metrics and incentives

Goodhart's law

- Goodhart's law: "When a measure becomes a target, it ceases to be a good measure."



Goodhart's law

The New York Times

The Price of Wells Fargo's Fake Account Scandal Grows by \$3 Billion

The bank reached a settlement with federal prosecutors and the Securities and Exchange Commission after abusing customers.

 Share full article    199



Wells Fargo used fraud to open up fake accounts and force customers into services

Incentivizing Productivity

➤ What happens when developer bonuses are based on

- Lines of code per day?
- Amount of documentation written?
- Low number of reported bugs in their code?
- Low number of open bugs in their code?
- High number of fixed bugs?
- Accuracy of time estimates?

What you need to know



Metrics are important in Software Engineering



Apply goal-oriented approaches to software metrics



Provide examples of metrics for software qualities and process



Understand limitations and dangers of decisions and incentives based on measurements



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.